University of BRISTOL

# Hardware Security

Andy Baldrian

bristol.ac.uk

# Content

- Rootkit
- Trusted Computing Base
- Trusted Platform Module
- Intel SGX
- ARM TrustZone



# Rootkit

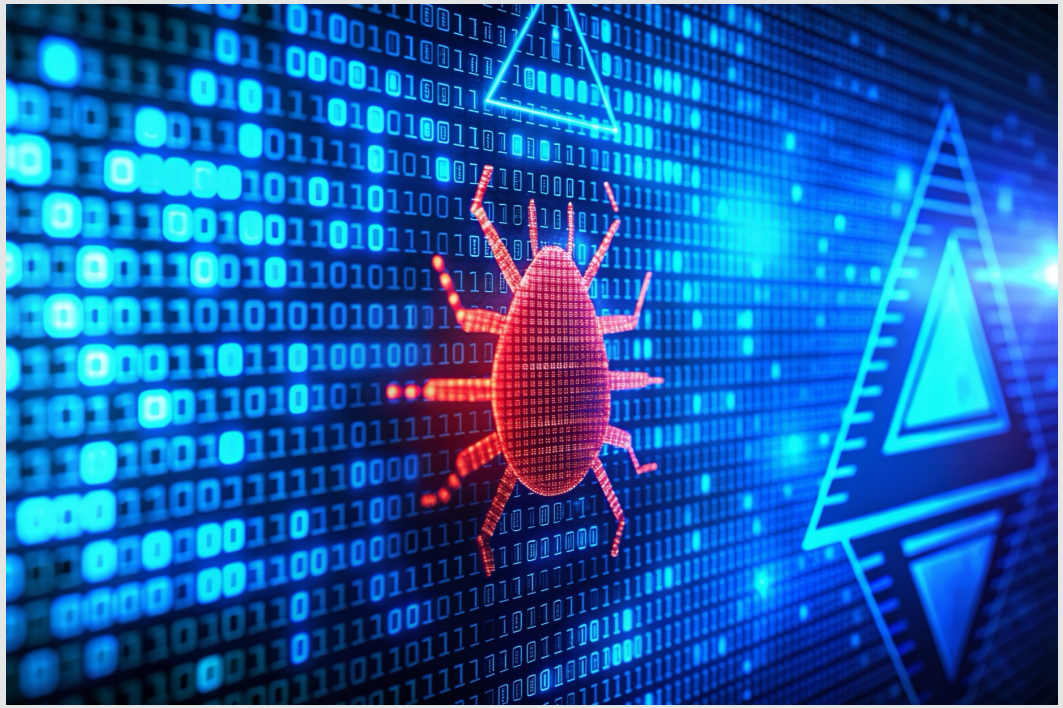


# Rootkit - Purpose



A kernel rootkit is malicious software that operates at the core of an operating system, remaining hidden, and giving full access to an attacker.

- Gives the attacker a permanent root access to a system
- Hides its presence
  - Hides from file system
  - Hides its activity
  - etc...
- Steals information
- Allows remote code execution



# Rootkit - Attacker steps

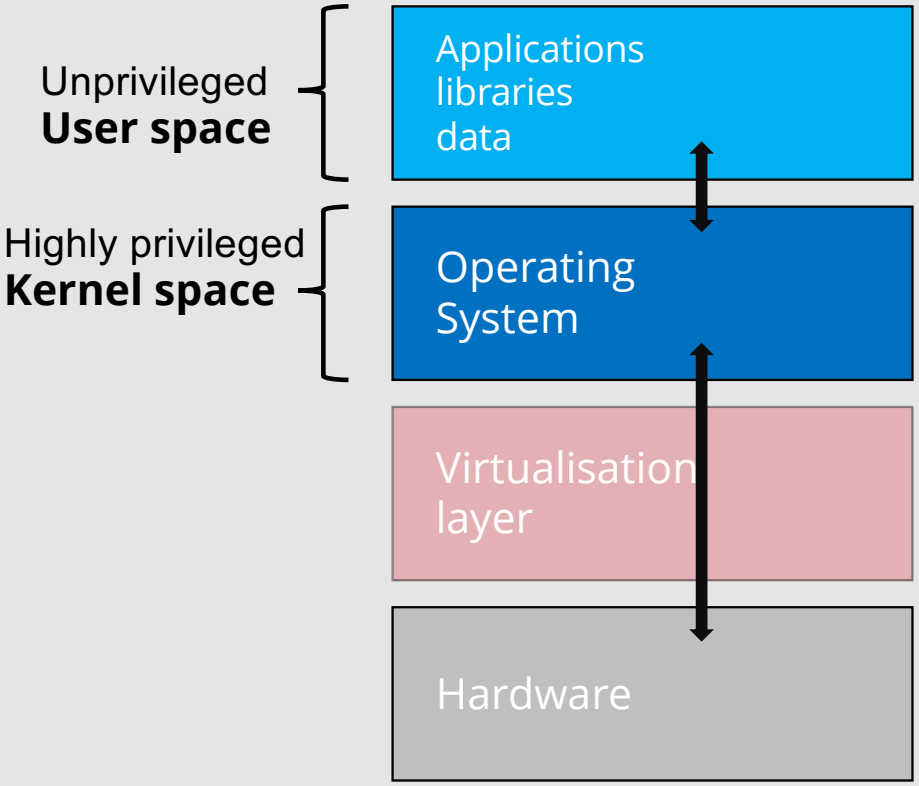





Cyber Kill Chain®, Lockheed Martin

- Initial intrusion (e.g. exploit remote execution)
- Open remote access (e.g. reverse shell)
- Privilege escalation
- Download the malicious payload (our rootkit)
- Install rootkit
- Perform malicious action on command
  - DDOS
  - Steal data
  - Generally, not being nice

# Kernel Rootkit - High-level understanding

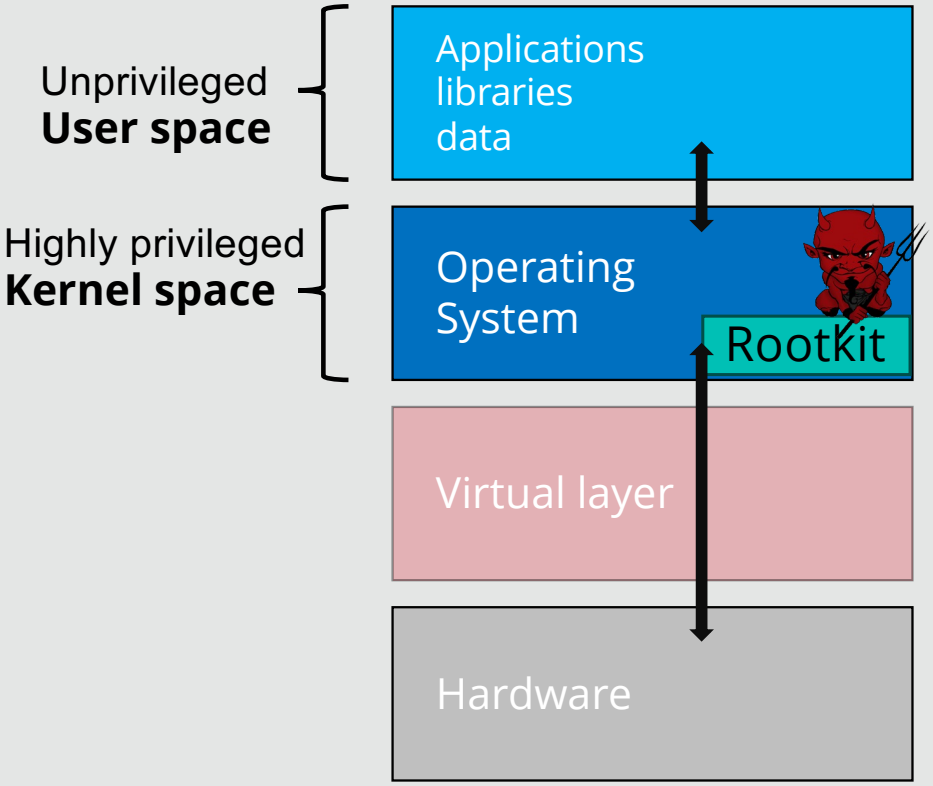



- Applications make a call to the Operating System (OS) via a **system call** to bridge the gap to kernel space.
- The Operating System (OS) runs at a higher privilege
  - Hardware access requires privileged instructions
  - Isolation and fault containment
- The Virtualisation Layer (optional) is an abstraction layer, enabling multiple virtual environments to run simultaneously on one physical machine.
- The hardware, CPU, memory, etc.

# Kernel Rootkit - High-level understanding

Rootkit

- Goal: hide malicious file/process etc.




Example

The user : `ubuntu@ubuntu:~$ ls`



- The root kit can intercept the `getdents()`* call and change what the user sees
- Modify returned value to exclude `./malicious/`

The disk returns:

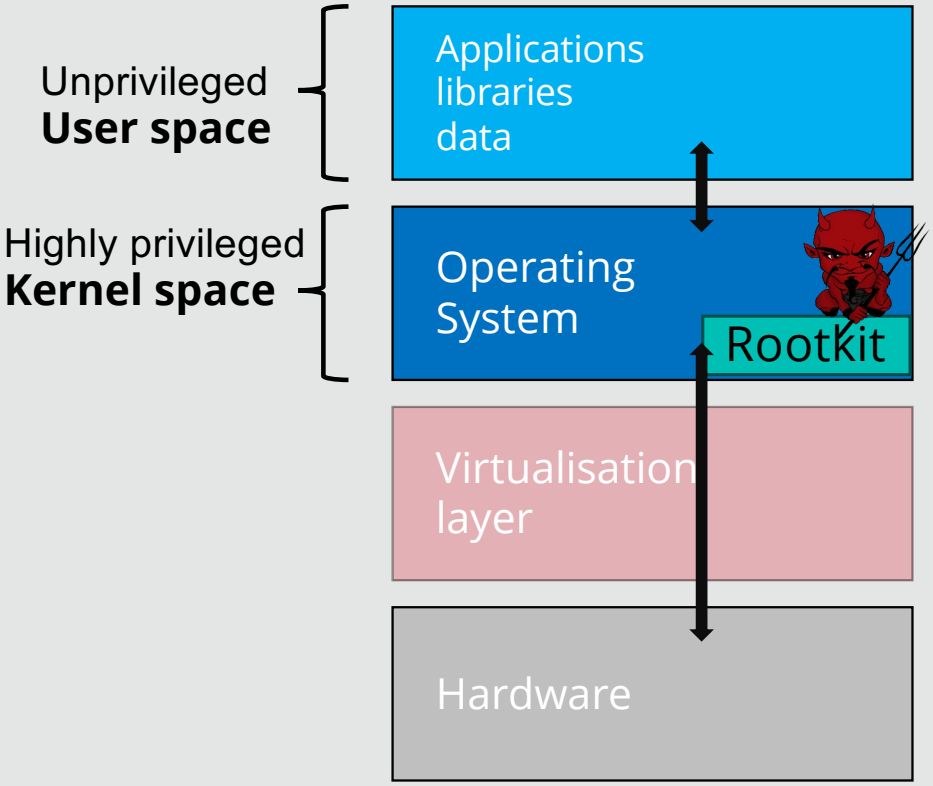
```
Desktop    Downloads  Pictures  Public
Templates  malicious
Documents  Music      Projects  Space
Videos     snap
```

* get directory entries
https://man7.org/linux/man-pages/man2/getdents.2.html

# Kernel Rootkit - High-level understanding

Rootkit

- Goal: hide malicious file/process etc.




Example

The user : `ubuntu@ubuntu:~$ ls`

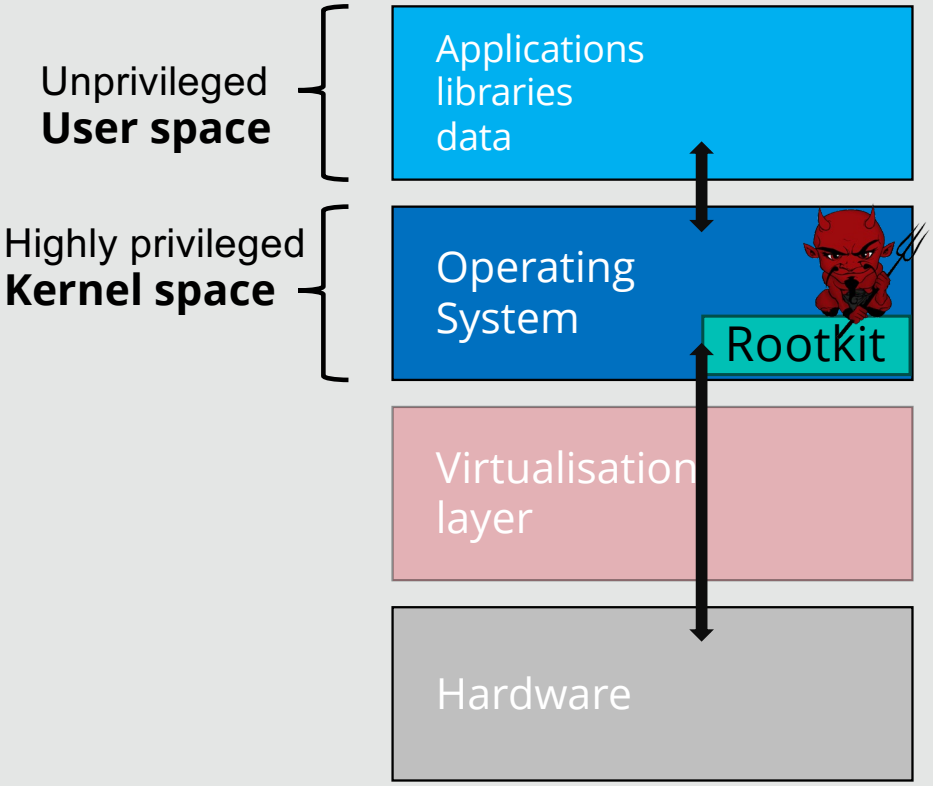
```
ubuntu@ubuntu:~$ ls
Desktop    Downloads  Pictures  Public
Templates  Documents  Music     Projects
Space      Videos     snap
```



The rootkit can modify the behaviour of anything that could reveal malware presence, e.g.:

`ls`
`ps`
`lsmod`
etc...

# Kernel Rootkit - High-level understanding




Roughly three techniques

- Modify the kernel code
- “Hooking”: modify where certain functions point to
- Modify data structure (e.g. active process list)

# Types of rootkit

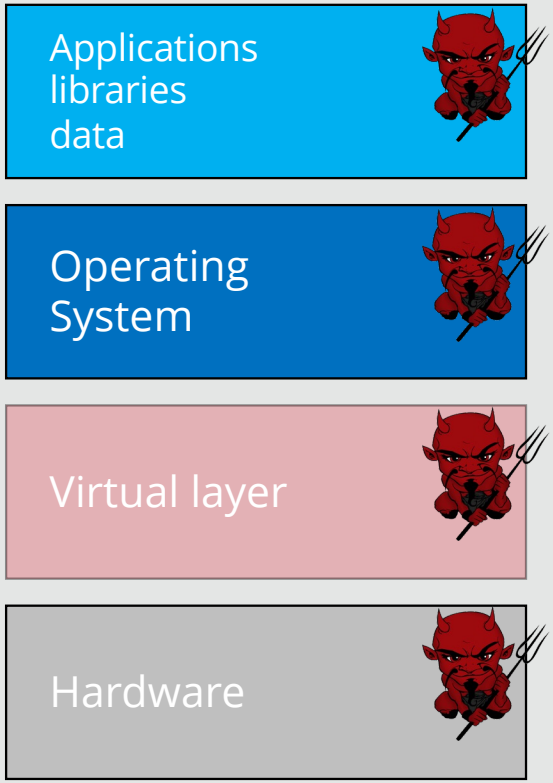


- Application rootkit
- Kernel rootkit
- Virtualised rootkit
- Bootloader rootkit
- Hardware & firmware rootkit

They can be prevented/detected by going (at least) one layer down

Examples:

- **Knark** and **Adore**, early Linux Loadable Kernel Module rootkits
- **Stuxnet**, which used a kernel-mode rootkit to sabotage industrial systems
- **ZeroAccess**, a botnet rootkit




Stühn, J., Hilgert, J. N., & Lambertz, M. (2024).
The hidden threat: Analysis of Linux rootkit techniques and limitations of current detection tools.

# Trusted Computing Base



# Attack Surface

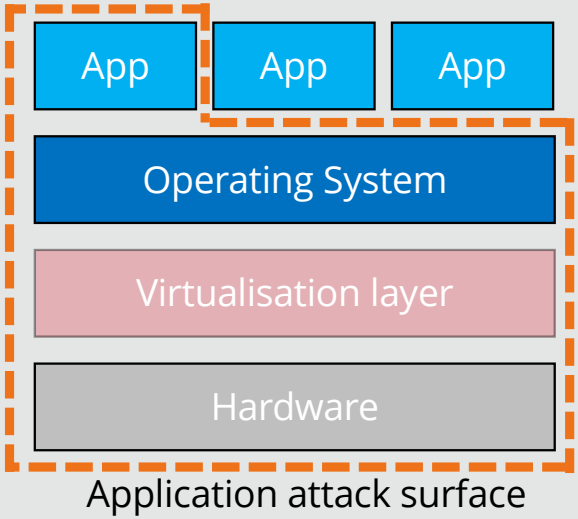




*"The set of points on the boundary of a system, a system element, or an environment where an attacker can try to enter, cause an effect on, or extract data from, that system, system element, or environment."*
NIST SP 800-53 Rev. 5

All the possible ways for an attacker to compromise a "system"

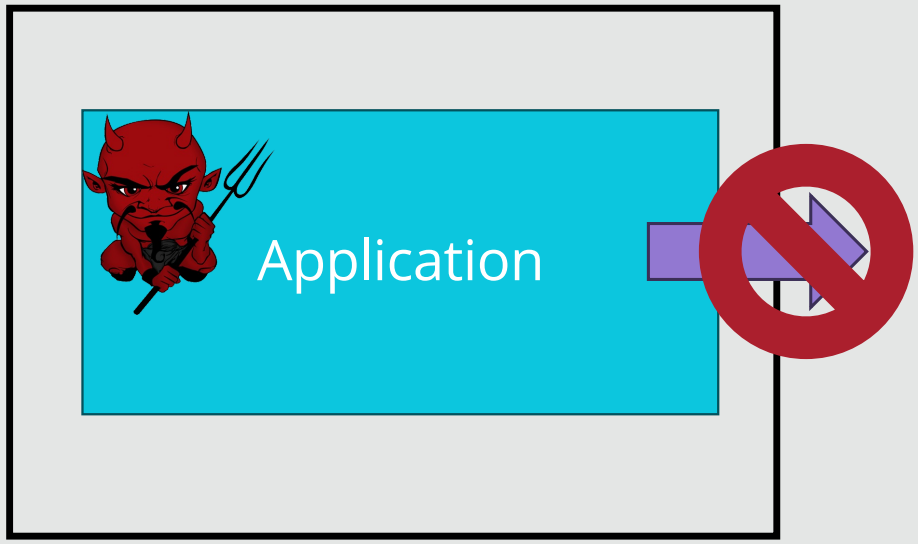
- Users
- Network
- Operating Systems
- Software
- Hardware
- etc.




How can we reduce the attack surface?

# Sandboxing






**Sandbox**

A restricted, controlled execution environment that prevents potentially malicious software from accessing any system resources except those for which the software is explicitly authorised

- Isolation
  - Strictly separated from the host machine
  - Preventing malware from accessing system resources, or other applications
- Containment
  - Malicious activity remains contained

Google Chrome

- Each tab and extension runs in its own sandbox

iOS

- Apps run in their own sandbox, preventing cross-app access

Adobe Reader (Protected Mode)

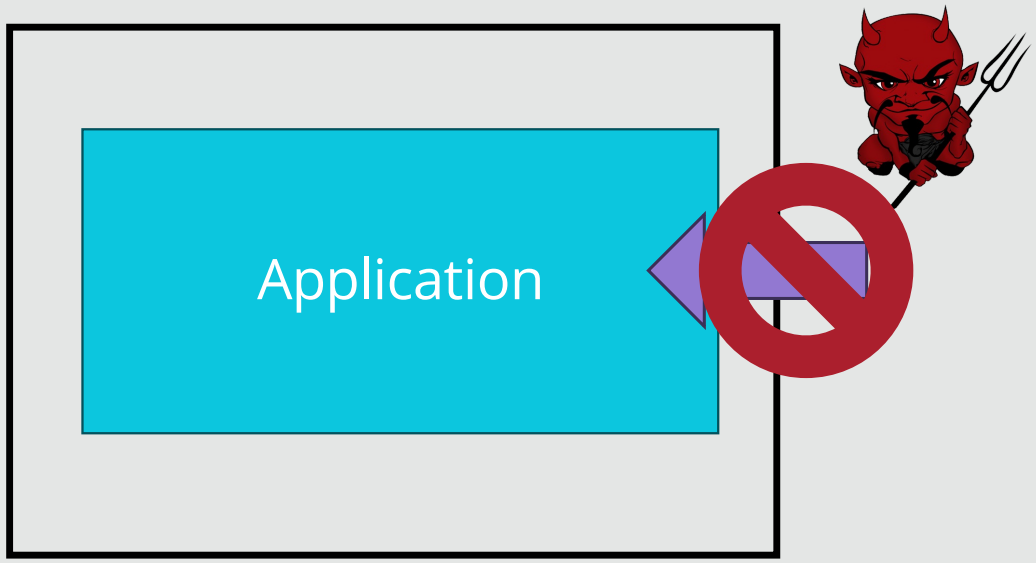
- Runs PDFs in a sandbox so embedded code can't escape the viewer

Cuckoo Sandbox

- Open-source automated malware analysis system

# Trusted Execution Environment






**Trusted Execution Environment (TEE)**
A hardware-isolated, secure area within a processor that guarantees the confidentiality and integrity of code and data executed inside it

- Hardware-enforced isolation
  - Partitions code execution into "Non-Trusted" and ”Trusted”.
- Integrity
  - Code running in the TEE cannot be modified or tampered with by external software

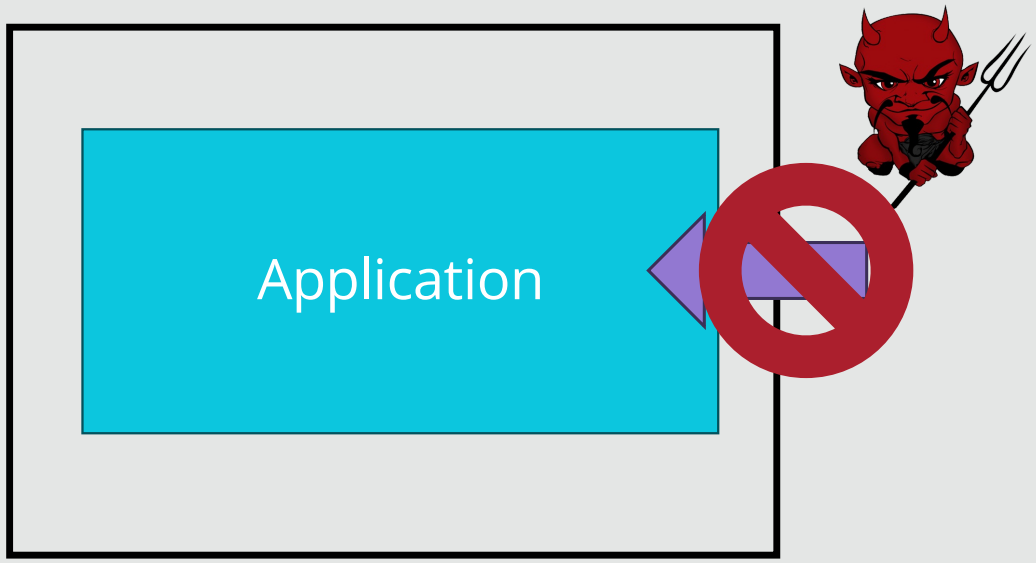
Intel SGX
- Application-level enclaves (up to ~256 GB)

ARM TrustZone
- System-wide secure world (mobile, IoT)

Apple Secure Enclave
- Dedicated co-processor for biometrics, keys

# Trusted Computing Base






*"a small amount of software and hardware that security depends on and that we distinguish from a much larger amount that can misbehave without affecting security."*
Lampson, Stoll, Massey, and Gifford (1992)
"Authentication in Distributed Systems: Theory and Practice"

Using the analogy of a bank:

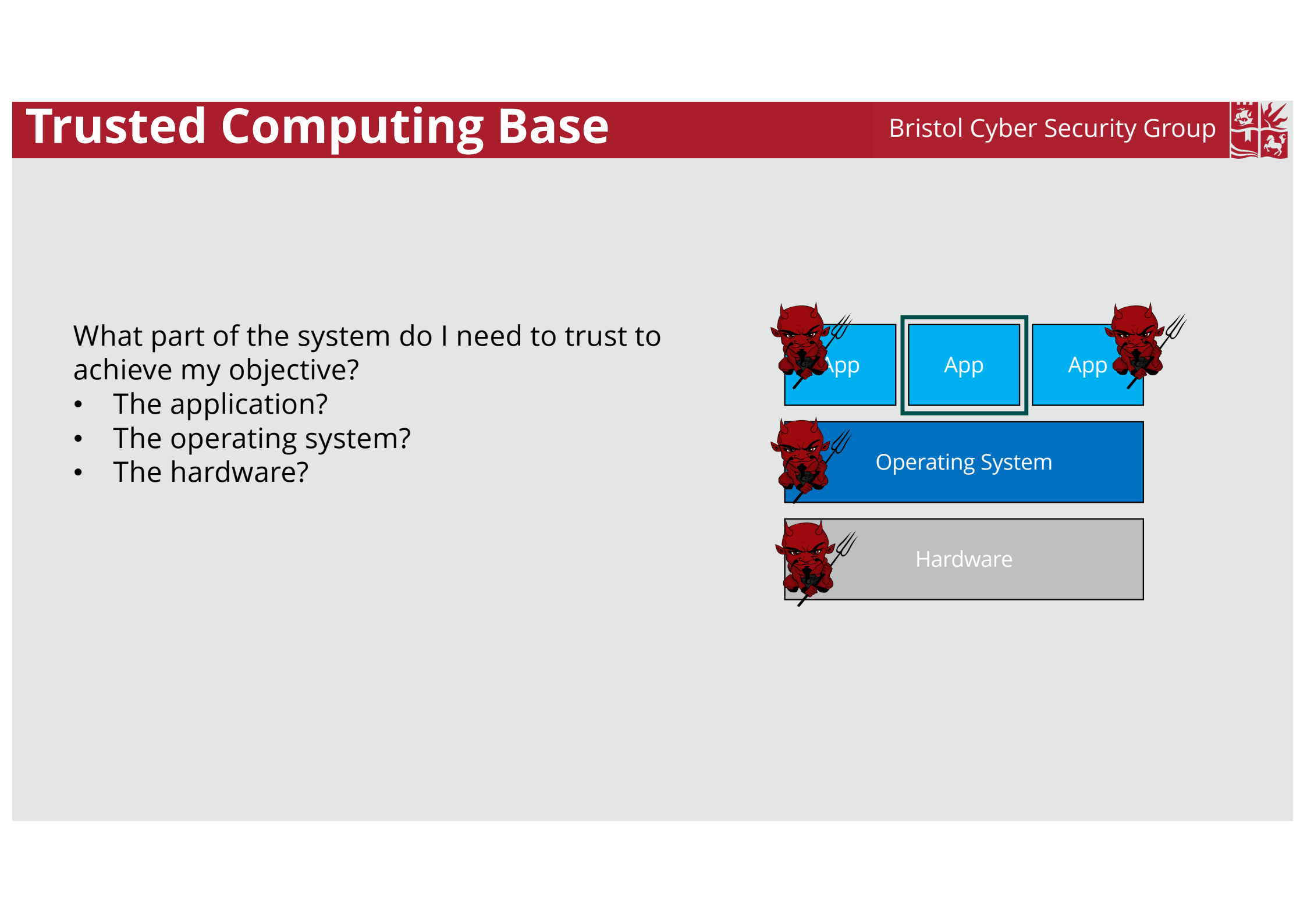
- A TEE is a **vault door:** a specific, physical object.
- A TCB is the **security perimeter** of the bank: the walls, the alarm, the guards, the policy, the vault door, the key custodian.

TCB is a **security architecture concept**

- Includes: hardware, firmware, software, people, and procedures
- The boundary drawn around the TCB is called the **security perimeter.**

# Trusted Computing Base





What part of the system do I need to trust to achieve my objective?

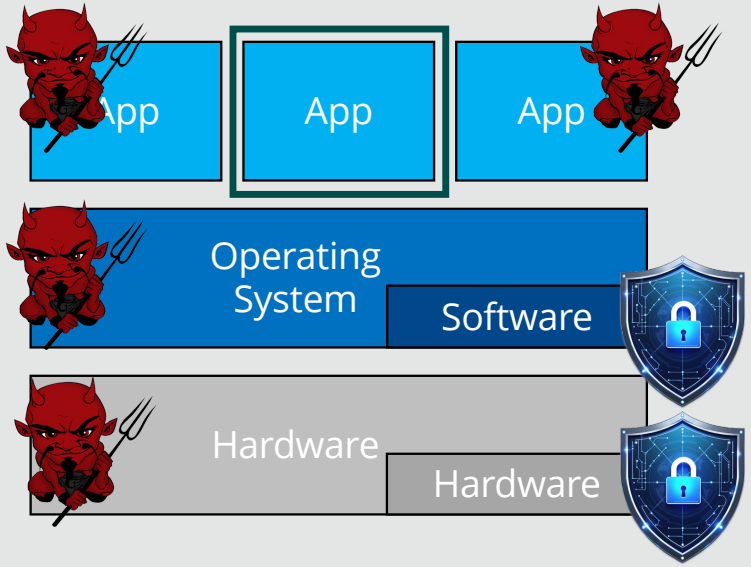
- The application?
- The operating system?
- The hardware?

# Trusted Computing Base



What part of the system do I need to trust to achieve my objective?

**Trusted Computing Base (TCB)**
"Totality of protection mechanisms within a computer system, including hardware, firmware, and software, the combination responsible for enforcing a security policy"
NIST SP 800-53




NIST Special Publication (SP) 800-53, Rev. 5. https://doi.org/10.6028/NIST.SP.800-53r5
Rushby, J. (1984). A trusted computing base for embedded systems.

# Trusted Platform Module



# Trusted Platform Module (TPM)






- Trusted Computing Group
  - Founded in 2003 by by AMD, HP, IBM, Intel, and Microsoft
  - Promoting standards for more trusted computing
  - Additional component on the motherboard: the TPM

- The TPM is used for:
  - Disk encryption
  - **System Integrity**
  - Password protection
  - … and more

# Trusted Platform Module (TPM)



Requirements

- We can achieve trust if we can verify the system has booted correctly
- We assume the PC hardware has not been modified
  - Key function is in the hardware TPM

- We need to monitor the boot process
  - Initial boot measure by the “Core Root of Trust”, Core Root of Trust for Measurement (CRTM)
  - Hash the BIOS, store results in TPM, start the BIOS
  - BIOS does its job, loads the next stage, the hash is stored in TPM



**Core Root of Trust:**
**“**The first piece of BIOS code that executes on the main processor during the boot process. On a system with a Trusted Platform Module the CRTM is implicitly trusted to bootstrap the process of building a measurement chain for subsequent attestation of other firmware and software that is executed on the computer system.”
NIST SP 800-147

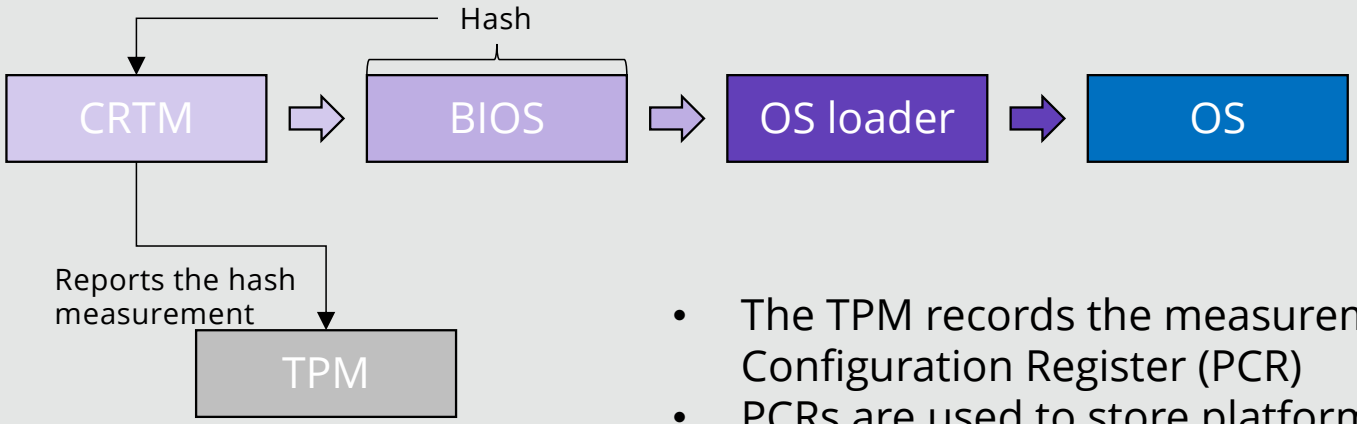
# Authenticated Boot





Core Root of Trust for Measurement (CRTM)

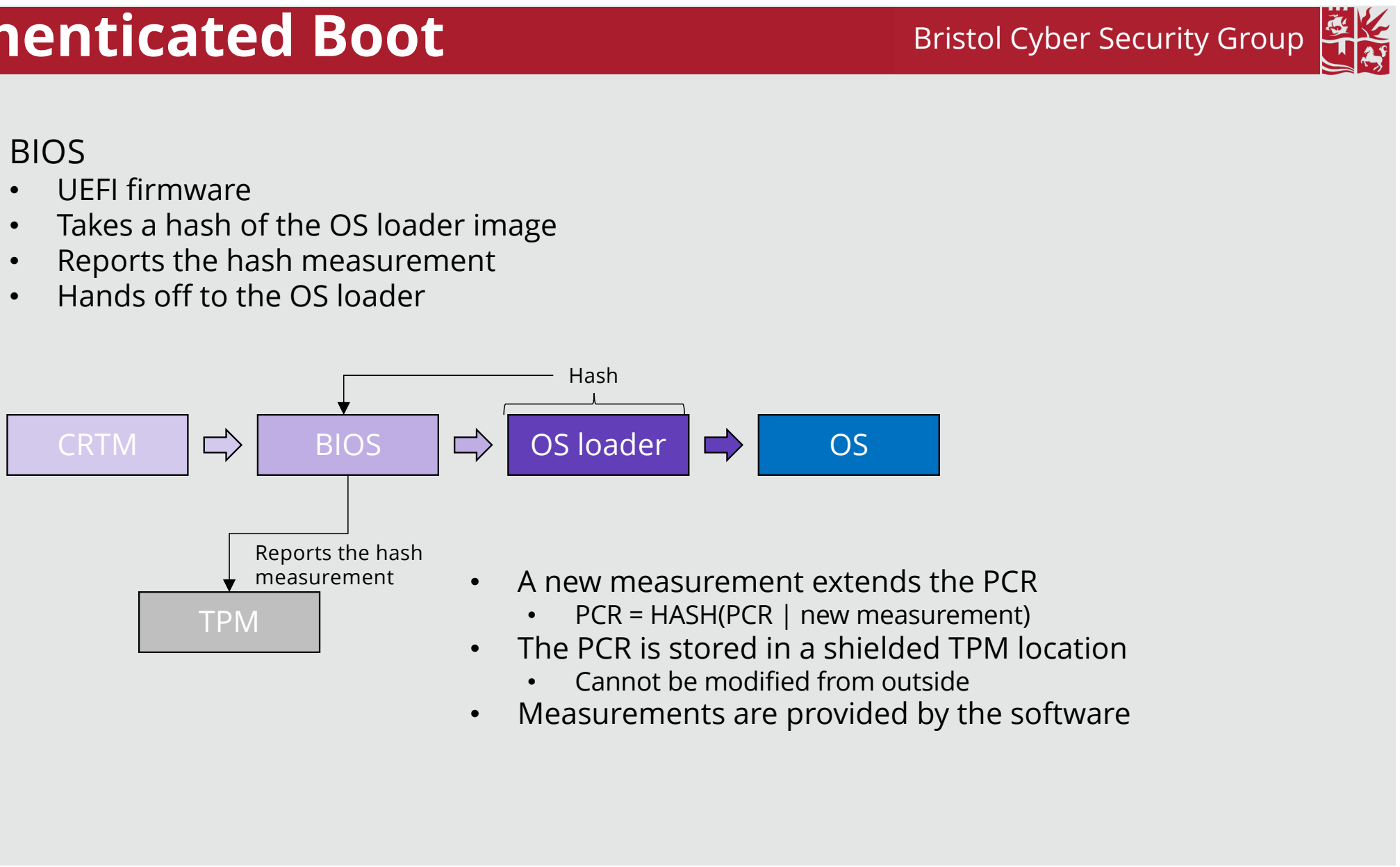
- A small, immutable piece of firmware (the BIOS/UEFI boot block)
- Takes a hash of the BIOS image
- Reports the hash measurement
- Hands off to the BIOS




- The TPM records the measurement in a Platform Configuration Register (PCR)
- PCRs are used to store platform integrity metrics
- A PCR holds a summary of a series of values
  - The chain can be infinite

# Authenticated Boot





BIOS
- UEFI firmware
- Takes a hash of the OS loader image
- Reports the hash measurement
- Hands off to the OS loader



- A new measurement extends the PCR
  - PCR = HASH(PCR | new measurement)
- The PCR is stored in a shielded TPM location
  - Cannot be modified from outside
- Measurements are provided by the software

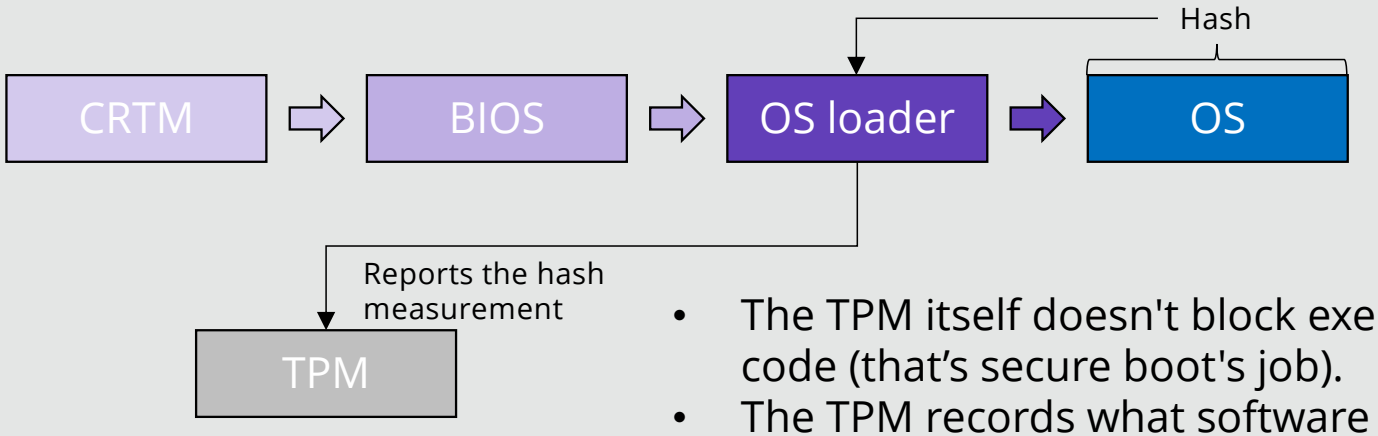
# Authenticated Boot





OS loader

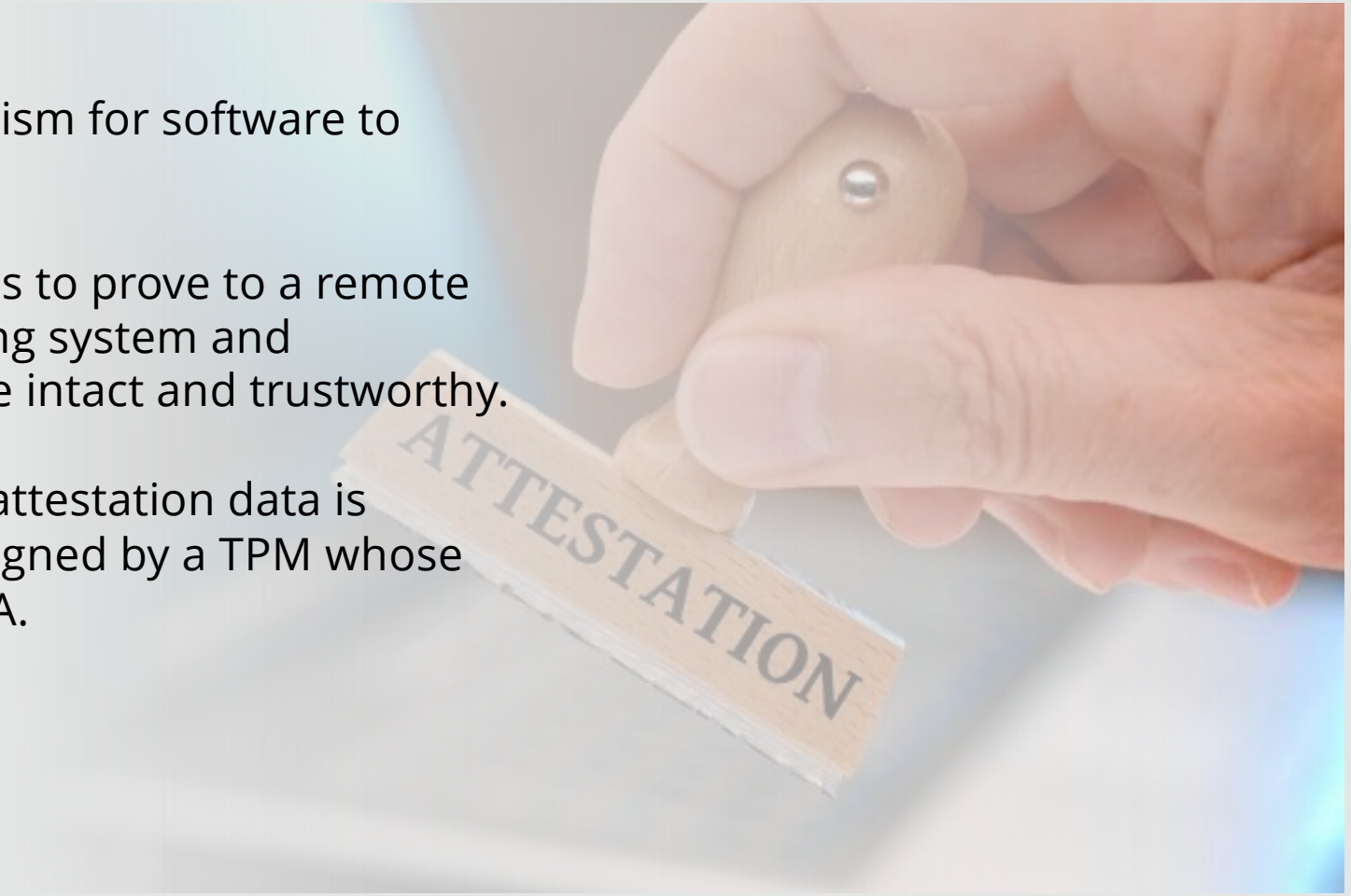
- e.g. GRUB
- Takes a hash of the OS image
- Reports the hash measurement
- Hands off to the OS




- The TPM itself doesn't block execution of untrusted code (that's secure boot's job).
- The TPM records what software is executed and provides cryptographic proof of it.
- PCRs cannot be modified
  - Only reset at reboot

# Remote attestation





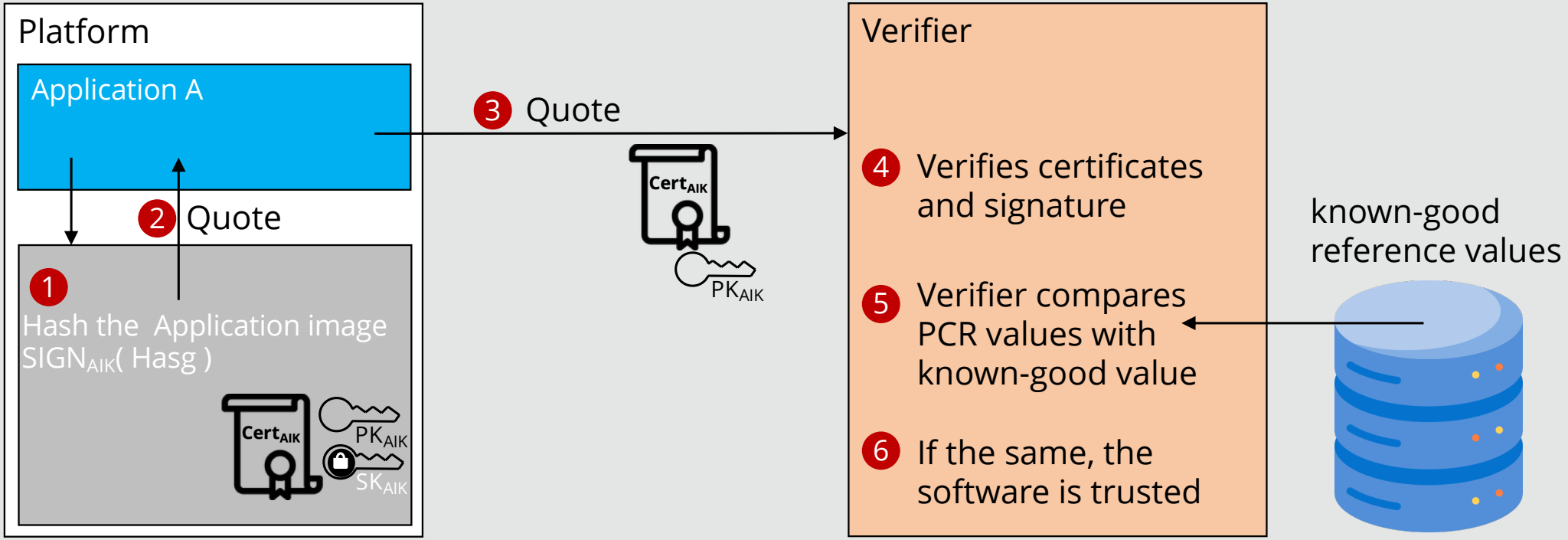

- Attestation is a mechanism for software to prove its identity.
- The goal of attestation is to prove to a remote party that your operating system and application software are intact and trustworthy.
- The verifier trusts that attestation data is accurate because it is signed by a TPM whose key is certified by the CA.

# Remote attestation : Example



Attestation:

1. Hash the application image
2. Creates Quote = $SIGN_{AIK}$( hash )
3. App sends: Quote + $Certificate_{AIK}$
4. Verifier checks: $Certificate_{AIK}$ → Quote signature
5. Verifier compares PCR values against known-good reference values
6. If match → trusted → proceed (e.g., key exchange)



# Remote attestation



The TPM contains two certificates each with a public and private key




Endorsement Key (EK) Certificate

- A unique asymmetric key pair burned into the chip at the factory.
- The certificate is signed using the manufacturer's certificate authority (CA)

What it proves: "This TPM is a genuine hardware TPM"




Attestation Identity Key (AIK) Certificate

- The asymmetric key pair is created *inside* the TPM
- Signed by the attestation CA
- Cryptographically bound to the EK Certificate using the proof-of-possession challenge.

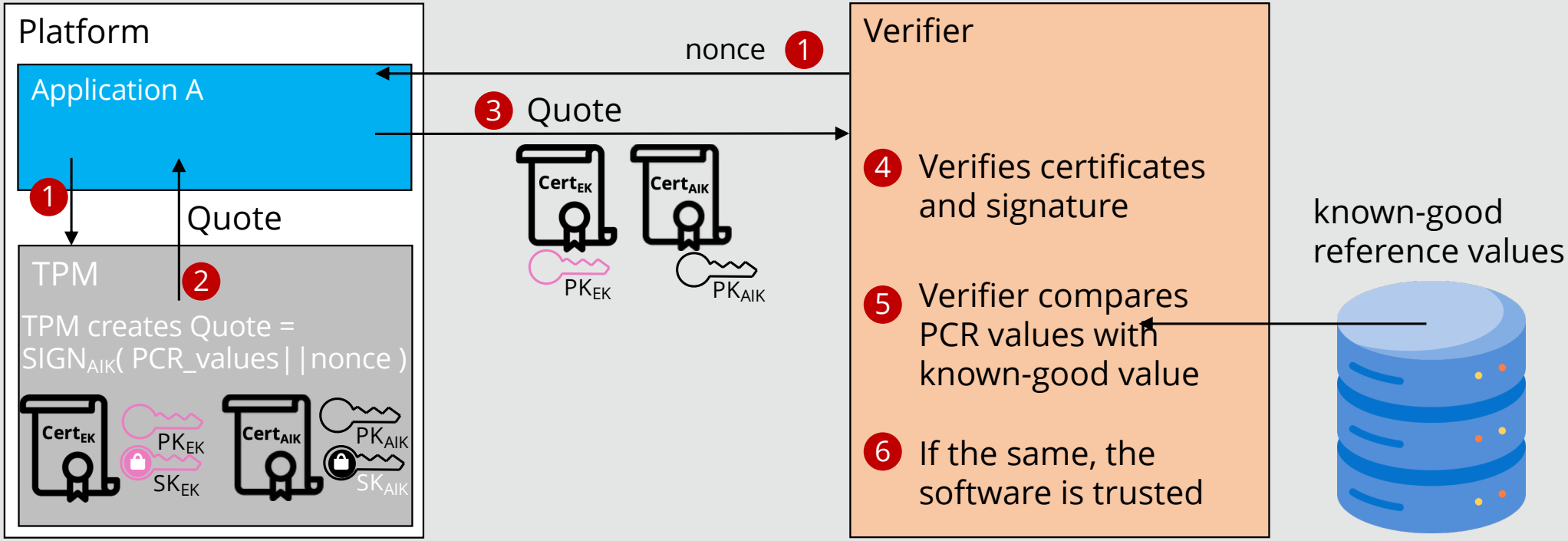
What it proves: " The TPM's Attestation Identity Key (AIK) can be trusted". (If you trust the attestation CA)

# Remote attestation : TPM



Attestation:

1. Remote party sends a nonce
2. TPM creates Quote = $SIGN_{AIK}$( PCR_values || nonce )
3. App sends: Quote + AIK certificate + TPM EK certificate
4. Verifier checks: EK cert → AIK cert → Quote signature
5. Verifier compares PCR values against known-good reference values
6. If match → trusted → proceed (e.g., key exchange)

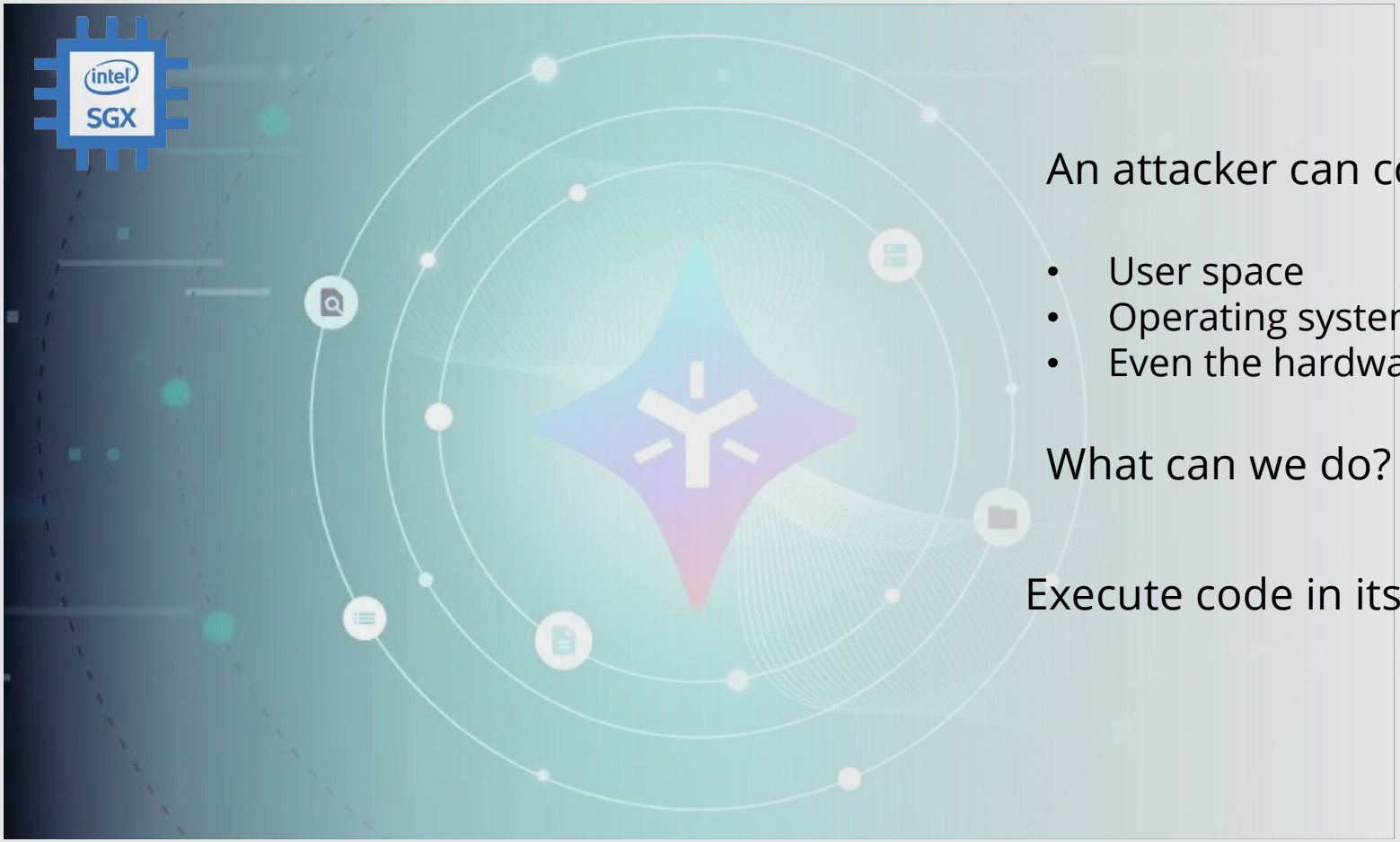


# Intel SGX



# Intel SGX: Motivation






An attacker can compromise:

- User space
- Operating systems
- Even the hardware!

What can we do?

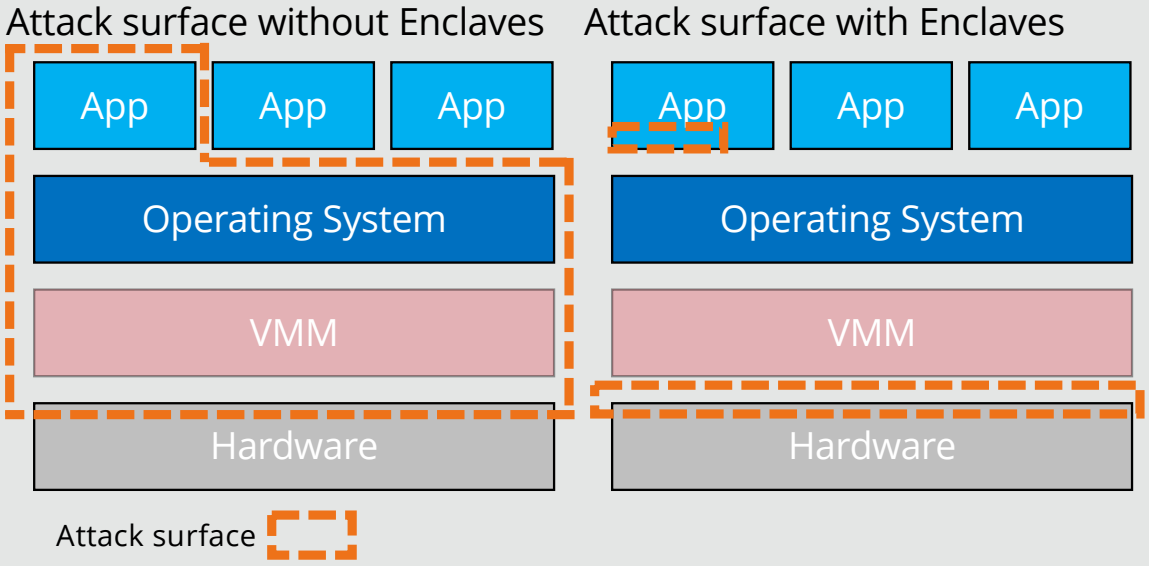
Execute code in its own secure enclave!

# SGX: Hardware supported enclave



**Idea**: Run an application within an isolation unit so it cannot be affected by the OS

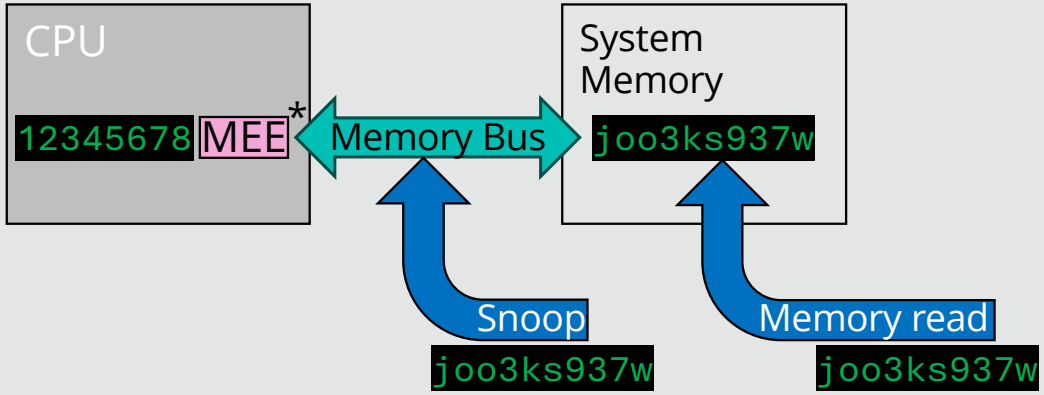
- Do not trust the OS or the virtual machine monitor(VMM)/hypervisor
- Only need to trust the hardware
- Reduces the attack surface



# SGX: Preventing memory snooping attack



- Security boundary is the CPU package
- Data outside the CPU is encrypted
- Data is unencrypted inside the CPU
- External memory reads and bus snooping only sees encrypted data

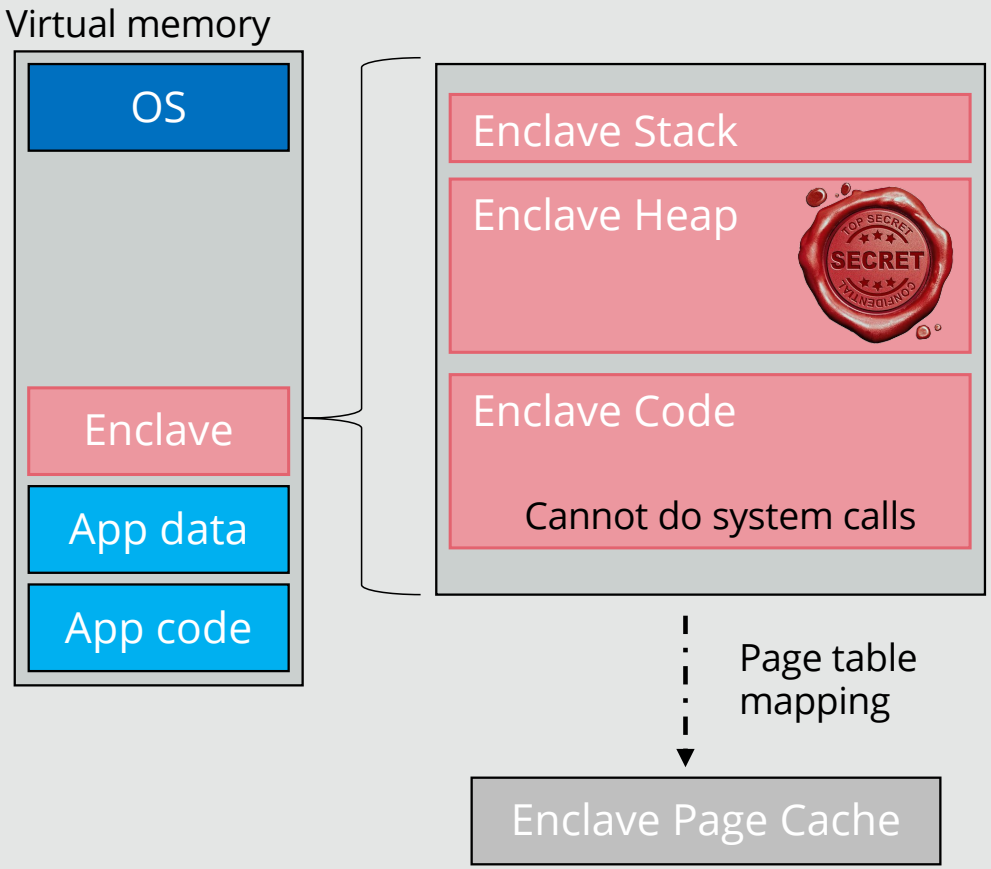



Costan, V., & Devadas, S. (2016). Intel SGX explained

* MEE: SGX Memory Encryption Engine

# SGX: Programming environment






Enclave has its own code and data

- Provides confidentiality and integrity

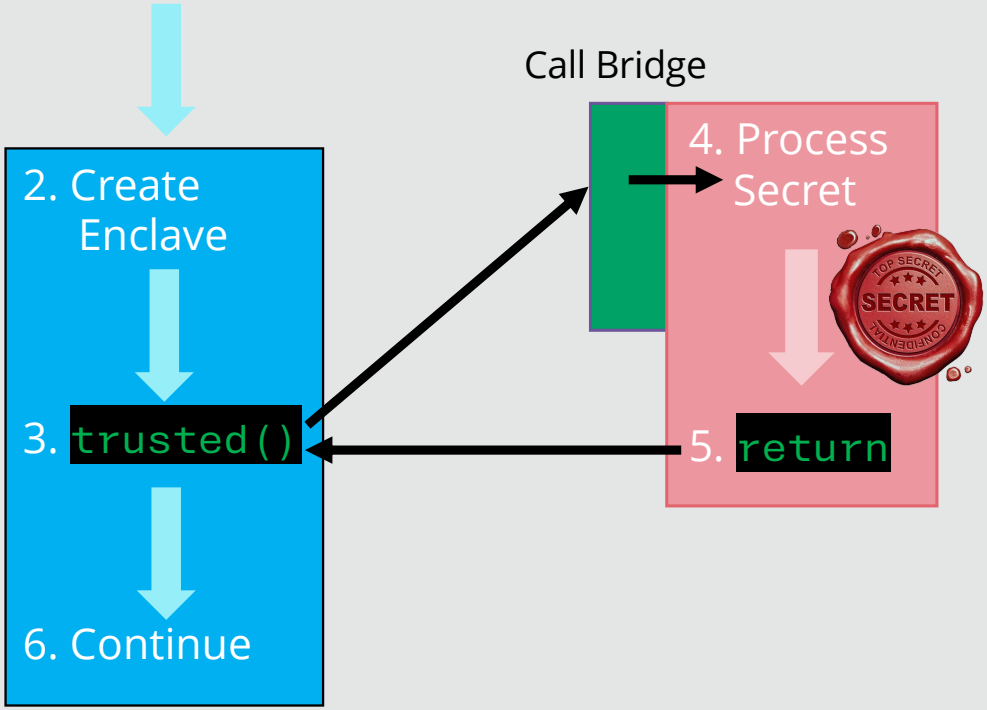
Controlled entry point

- Can enter enclave code only at a specific point
- Enclave execution takes over

# SGX: Application flow



1. Define and divide the application into trusted and untrusted parts
2. The application creates enclave
3. The `trusted()` function is called
4. Code in enclave processes some secret
5. The `trusted()` function returns
6. The app continues as normal



# ARM TrustZone

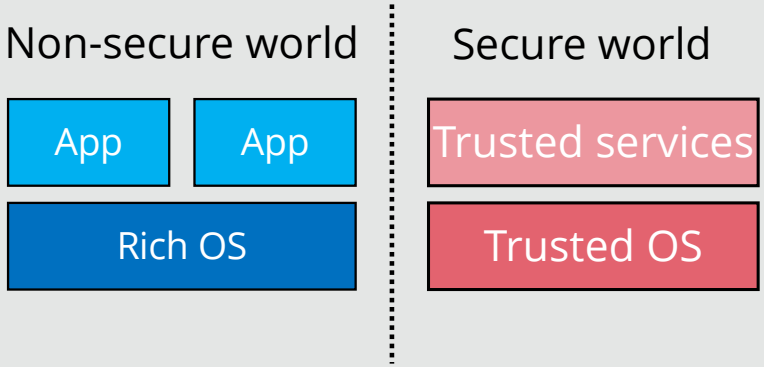


# ARM TrustZone



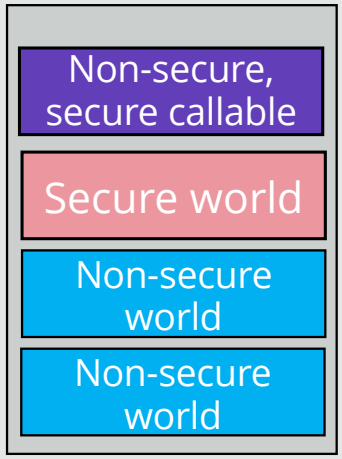
The CPU operates in two modes:

- Normal (non-secure world)
- Trusted (secure world)

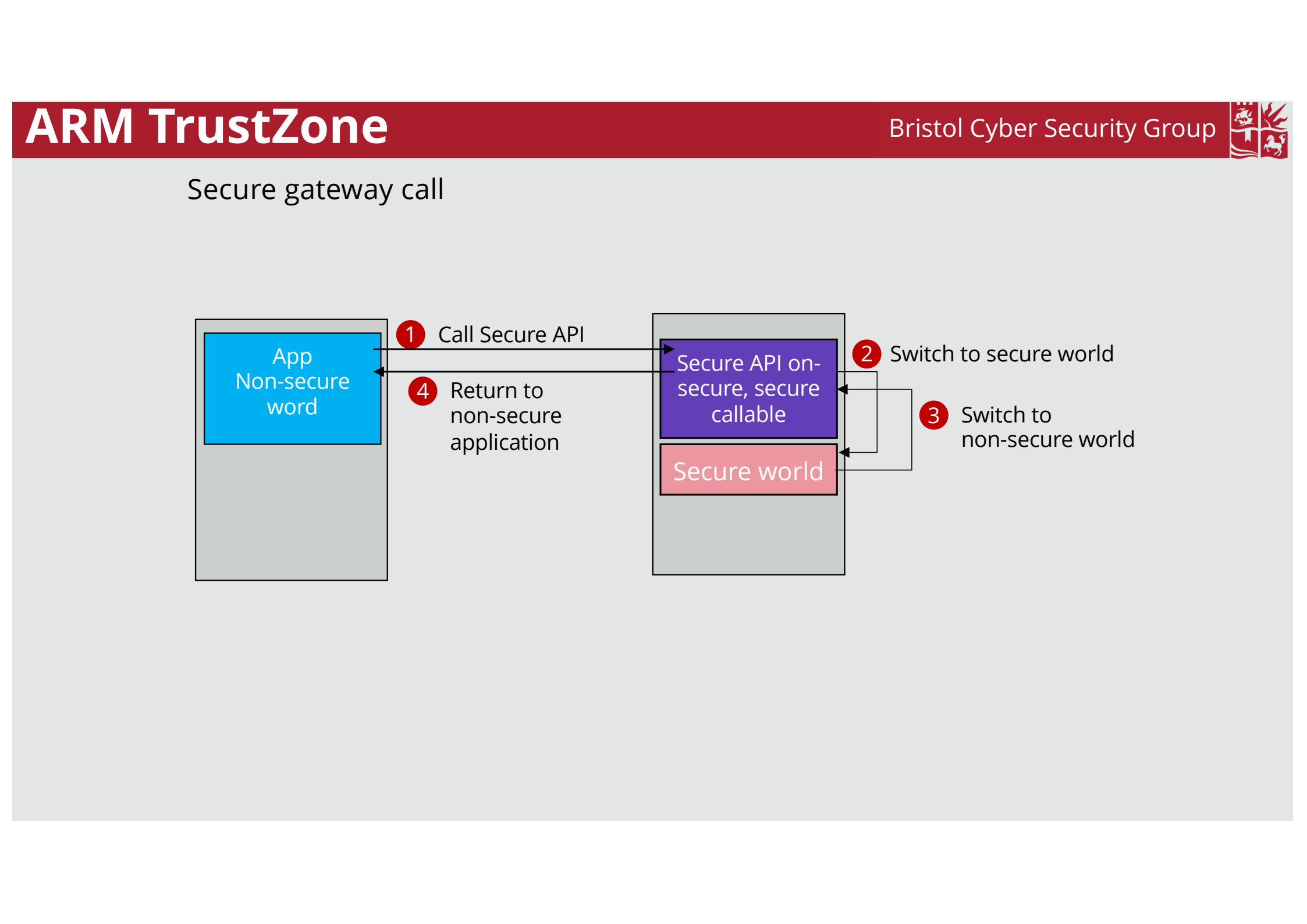



Security Attribution Unit (SAU)
Memory is split into:

- Non-secure world
- Secure world
- Non-secure, secure callable




Yuhala, P., Ménétrey, J., Felber, P., Pasin, M., & Schiavoni, V. (2024, April). Fortress: Securing iot peripherals with trusted execution environments.
Ngabonziza, B., Martin, D., Bailey, A., Cho, H., & Martin, S. (2016, November). Trustzone explained: Architectural features and use cases.



## Secure gateway call

# ARM TrustZone: Trusty OS



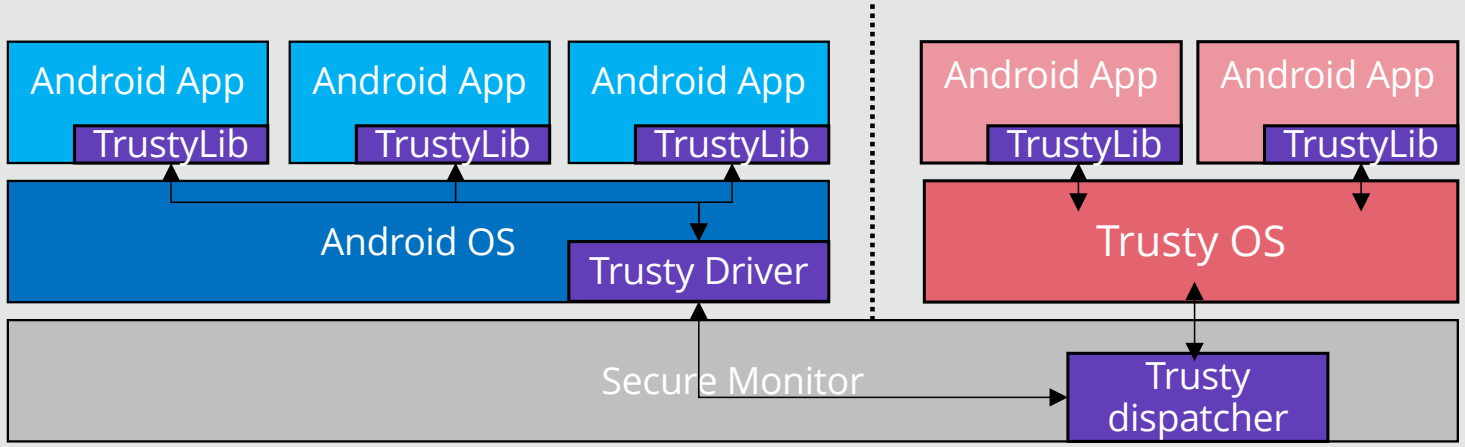
Examples:

Digital Rights management

Secure banking

Multi-factor authentication

etc...



# Conclusion: Reduce the attack surface

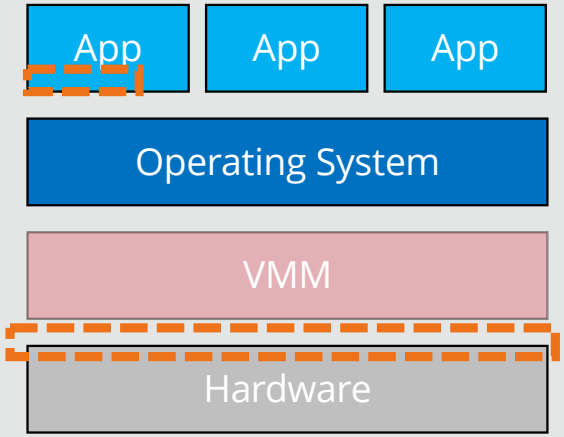


Attestation
- Test and assert the software that is running

Sandbox
- Isolation and containment

Trusted Execution Environment
- Hardware-enforced isolation and integrity

Trusted Computing Base
- Includes: hardware, firmware, software, people, and procedures
- Minimum functionality required



# Questions?



